

PSYC 51.17: Models of Language and Communication

# Lecture 18: BERT Deep Dive

Week 6, Lecture 1 - Bidirectional Encoder  
Representations

PSYC 51.17: Models of Language and Communication

Winter 2026

Winter 2026

# Today's Agenda

1. **BERT Introduction:** What makes it special?
2. **Masked Language Modeling:** The key training objective
3. **BERT Architecture:** Model sizes and specifications
4. **Pre-training & Fine-tuning:** The two-stage paradigm
5. **Contextual Embeddings:** Seeing polysemy in action
6. **Using BERT:** Practical code examples

*Goal: Deep understanding of BERT and how it revolutionized NLP*

Winter 2026

# BERT: Bidirectional Encoder Representations

**BERT = Encoder-only Transformer**

**Key Innovation: Masked Language Modeling (MLM)**

**Traditional Language Models:**

- Left-to-right (GPT)
- Or right-to-left
- Can't see full picture

**BERT (MLM):**

- Mask random tokens
- Predict from
- Sees both left & right

Deeper understanding

## **1. Deep Bidirectionality**

- True bidirectional context at every layer
- Not just concatenating left-to-right and right-to-left

## **2. Pre-train + Fine-tune Paradigm**

- Single pre-trained model for all tasks
- Fine-tune with minimal architecture changes

# Masked Language Modeling (MLM)

## BERT's Pre-training Objective

### Training Procedure:

1. Take a sentence
2. Randomly mask 15% of tokens
3. Of the masked tokens:
  - 80%: Replace with [MASK]
  - 10%: Replace with random word
  - 10%: Keep unchanged

## MLM Example: Step by Step

**Sentence:** "The quick brown fox jumps over the lazy dog"

### Step 1: Select Tokens (15%)

9 tokens total, mask ~1-2

1

```
tokens =  
["The",  
"quick",  
"brown"]
```

### Step 3: Create Training Example

1

```
input: "The  
[MASK] brown  
fox jumps  
under the
```



# Next Sentence Prediction (NSP)

**BERT's second pre-training objective (debated usefulness)**

**Task:** Given two sentences A and B, predict if B follows A

Positive Example (IsNext)

**Sentence A:** "The man went to the store."

**Sentence B:** "He bought a gallon of milk."

**Label:** IsNext

Negative Example (NotNext)

**Sentence A:** "The man went to the store."

**Sentence B:** "Penguins are flightless birds."

**Label:** NotNext

## BERT Architecture Variants

|                        |           |             |           |             |
|------------------------|-----------|-------------|-----------|-------------|
| <b>BERT-<br/>Large</b> | <b>24</b> | <b>1024</b> | <b>16</b> | <b>340M</b> |
|------------------------|-----------|-------------|-----------|-------------|

### Architecture Details (BERT-Base):

- 12 transformer encoder layers
- 768-dimensional hidden states
- 12 attention heads per layer (64 dims each)
- 3072-dimensional feed-forward intermediate size (4x



# BERT Input Representation

Three types of embeddings are summed:

```
1 | # Example: Sentence pair for NSP
```

**Three embedding types:**

1. **Token Embeddings:** WordPiece vocabulary (30K learned vectors)
2. **Segment Embeddings:** Which sentence (A or B)? (2 learned vectors)
3. **Position Embeddings:** Learned position 0-511 (512 learned vectors)

# WordPiece Tokenization: Worked Example

## How BERT handles unknown words

```
1 | from transformers import BertTokenizer
```

### Why WordPiece?

- **No OOV problem:** Any word can be represented as subwords
- **Morphology:** Learns word parts (prefixes, suffixes, stems)
- **Compact vocabulary:** 30K tokens cover most text
- Trade-off: Rare words take more tokens (longer sequences)

## BERT Pre-training

**Massive scale pre-training on unlabeled text**

### **Pre-training Data:**

- **BooksCorpus:** 800M words (novels, fiction)
- **English Wikipedia:** 2,500M words
- Total: 3.3 billion words
- Diverse, high-quality text

### **Training Details:**

# Fine-tuning BERT

**Two-stage process: Pre-train then Fine-tune**

**Stage 1: Pre-training** (done once)

- |   |                                              |
|---|----------------------------------------------|
| 1 | # Expensive:<br>weeks on TPUs                |
| 2 | # Data: 3.3B<br>words (books +<br>Wikipedia) |

**Stage 2: Fine-tuning** (per task)

- |   |                                 |
|---|---------------------------------|
| 1 | # Cheap: hours on               |
| 2 | # Data: 1K-100K 1               |
| 3 | # Task: Your spec <sup>12</sup> |
| 4 | # Result: Task-sp               |
| 5 |                                 |

## 2. Sentence Pair Classification

- Input: [CLS] sentence A [SEP] sentence B [SEP]
- Output: [CLS] representation → classifier
- Example: Natural Language Inference

## 3. Question Answering

- Input: [CLS] question [SEP] passage [SEP]
- Output: Token-level predictions for start/end positions

# Fine-tuning BERT: Code Example

## Using HuggingFace Transformers

```
1 from transformers import BertForSequenceClassification, Trainer, TrainingArguments
2
3 # Load pre-trained BERT with classification head
4 model = BertForSequenceClassification.from_pretrained(
5     'bert-base-uncased',
6     num_labels=2 # Binary classification
7 )
8
9 # Define training arguments
10 training_args = TrainingArguments(
11     output_dir='./results',
12     num_train_epochs=3,
13     per_device_train_batch_size=16,
14     learning_rate=2e-5,
15     warmup_steps=500,
16 )
17
18 # Train
19 trainer = Trainer(
```

14



## Fine-tuning BERT: Code Example

```
20 | model=model,  
21 | args=training_args,  
22 | train_dataset=train_dataset,  
23 | eval_dataset=eval_dataset,  
24 | )  
25 |  
26 | trainer.train()
```

*Reference: HuggingFace Course - Chapters 1.5, 7.3*

Winter 2026

# Contextual Embeddings in Action

**Remember "bank"? Let's see BERT handle it!**

```
1 from transformers import BertTokenizer, BertModel
2 import torch
3
4 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
5 model = BertModel.from_pretrained('bert-base-uncased')
6
7 # Two different contexts for "bank"
8 sent1 = "I deposited money at the bank"
9 sent2 = "We sat by the river bank"
10
11 # Get embeddings
12 def get_embedding(sentence, target_word):
13     inputs = tokenizer(sentence, return_tensors='pt')
14     outputs = model(**inputs)
15     # Find position of target word
```

16

Winter 2026

continued...

## Contextual Embeddings in Action

```
16 tokens = tokenizer.tokenize(sentence)
17 idx = tokens.index(target_word) + 1 # +1 for [CLS]
18 return outputs.last_hidden_state[0, idx, :]
19
20 emb1 = get_embedding(sent1, "bank") # Financial bank
21 emb2 = get_embedding(sent2, "bank") # River bank
22
23 # Compare similarity
24 similarity = torch.cosine_similarity(emb1, emb2, dim=0)
25 print(f"Similarity: {similarity:.3f}") # Low! (~0.3-0.5)
26 # Different contexts → Different embeddings!
```

Winter 2026

17

...continued

# Visualizing BERT's Contextual Embeddings

**Same word, different meanings, different vectors**

```
1 # Find nearest neighbors for "bank" in each context
```

## Concrete Measurements

| Word Pair                  | Word2Vec Similarity | BERT Similarity |
|----------------------------|---------------------|-----------------|
| bank (fin) vs bank (river) | 1.00 (same vector!) | 0.42            |
| bank (fin) vs money        | 0.65                | 0.78            |
| bank (river) vs shore      | 0.52                | 0.81            |

BERT captures meaning differences that static embeddings miss!

## BERT's Impressive Results

**State-of-the-art on 11 NLP tasks when released  
(2018)**

|                           |           |             |             |
|---------------------------|-----------|-------------|-------------|
| <b>SQuAD 2.0<br/>(QA)</b> | <b>F1</b> | <b>66.3</b> | <b>83.1</b> |
| MNLI (NLI)                | Accuracy  | 80.6        | 86.7        |
| SST-2<br>(Sentiment)      | Accuracy  | 93.2        | 94.9        |

## 2. Semantic Information

- Word sense disambiguation
- Semantic roles
- Entity types
- Middle layers encode more semantics

## 3. Pragmatic Information



# BERT Layer Analysis

## Different layers capture different linguistic properties

1 | # Probing experiment • Train linear classifiers on each layer's

### Observations:

- Lower layers: surface features (word forms, POS)
- Middle layers: syntax (phrase structure, dependencies)
- Higher layers: semantics and task-specific features

*Similar to CNNs: edges → shapes → objects*

Winter 2026

- Why not just use 100% [MASK]?
- What problem does the random replacement solve?
- Could we improve this strategy?

### **3. Pre-training Data:**

- Why use books and Wikipedia?
- Would social media text work as well?

- Masked Language Modeling
- Pre-training and fine-tuning
- Contextual embeddings
- What BERT learns

## **Next lecture (Lecture 16 - BERT Variants):**

- : Optimized BERT training
- : Parameter-efficient BERT
- : Smaller, faster BERT
- : ELECTRA, DeBERTa, and more

- Cheap fine-tuning on task-specific data (per task)

### **3. Contextual Embeddings**

- Different representations based on context
- Solves polysemy problem

### **4. Hierarchical Learning**

- Introduced MLM and NSP

\item **Tenney et al. (2019)** - "BERT Rediscovered the Classical NLP Pipeline"

- Analysis of what BERT learns
- Layer-wise linguistic properties

\item **Clark et al. (2019)** - "What Does BERT Look At? An Analysis of BERT's Attention"

- Understanding BERT's attention patterns

# Questions:

## Discussion Time

### Topics for discussion:

- Masked Language Modeling
- Pre-training vs fine-tuning
- Contextual embeddings
- BERT architecture details
- Implementation questions

Thank you!

26

Next: BERT Variants and Improvements!

Winter 2026